

Name: Bùi Quang Minh
Class: DS66B

Lab Exercise 1: Triggering Nested Loop Join

SQL:

EXPLAIN (ANALYZE, BUFFERS)

SELECT c.first_name, c.last_name, r.rental_date

FROM customer c

JOIN rental r ON c.customer_id = r.customer_id

WHERE c.customer_id = 1;

The screenshot shows the pgAdmin 4 interface. On the left, the 'Object Explorer' pane shows the database structure, with the 'customer' table selected. The main pane displays the SQL query and its execution plan. The query is:

```
1 EXPLAIN (ANALYZE, BUFFERS)
2 SELECT c.first_name, c.last_name, r.rental_date
3 FROM customer c
4 JOIN rental r ON c.customer_id = r.customer_id
5 WHERE c.customer_id = 1;
```

The execution plan shows a Nested Loop join. The plan steps are:

- Nested Loop (cost=0.28..359.16 rows=32 width=21) (actual time=0.495..4.339 rows=35.00 loops=1)
- Buffers: shared hit=153
- > Index Scan using customer_pkey on customer c (cost=0.28..8.29 rows=1 width=17) (actual time=0.431..0.434 rows=1.00 loops=1)
- Index Cond: (customer_id = 1)
- Index Searches: 1
- Buffers: shared hit=3
- > Seq Scan on rental r (cost=0.00..350.55 rows=32 width=10) (actual time=0.060..3.887 rows=35.00 loops=1)
- Filter: (customer_id = 1)
- Rows Removed by Filter: 16012
- Buffers: shared hit=150

Total rows: 14. Query complete 00:00:00.149. The status bar shows 'CRLF Ln 5, Col 26'.

Identify the Roles:

The customer table acts as the outer table because it is scanned first at the top of the plan tree. The rental table acts as the inner table.

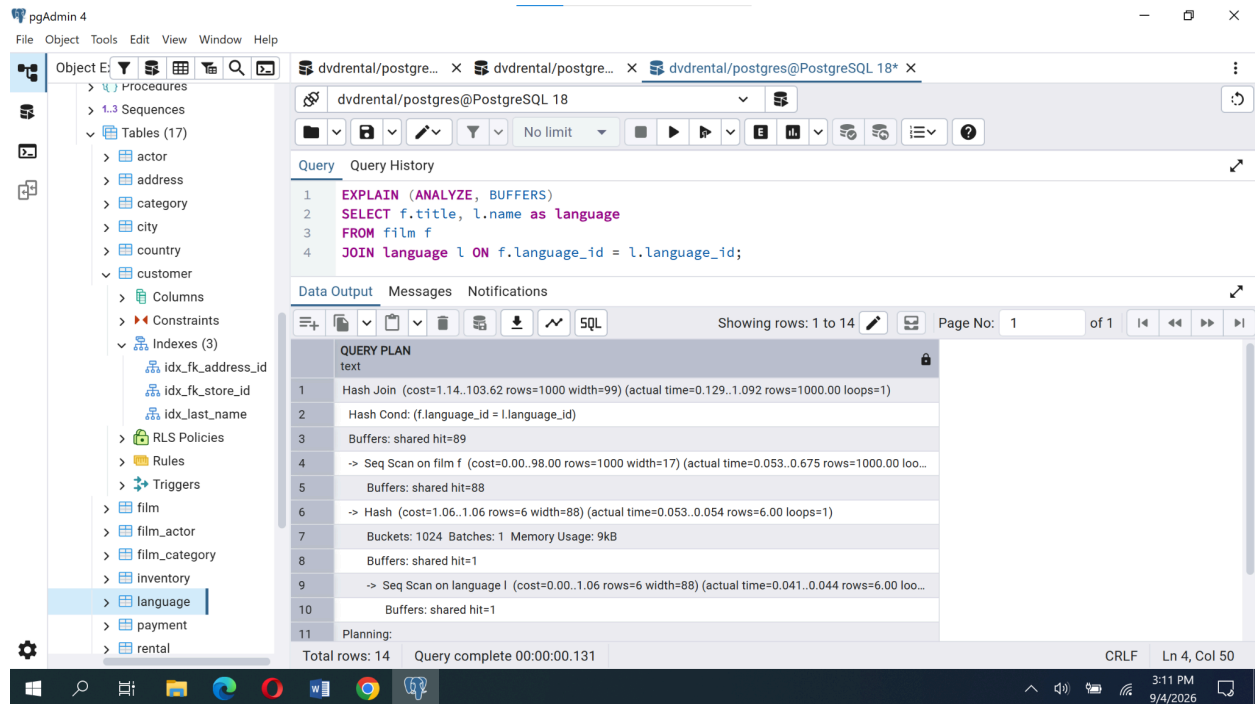
Why Nested Loop?:

The filter condition `c.customer_id = 1` restricts the customer table to return exactly 1 row. Building an entire hash table in memory for a single row is computationally expensive and inefficient compared to executing a direct inner lookup.

Efficiency:

The loops=1 metric on the inner table node proves that the database only had to execute the inner scan exactly one time. This high efficiency occurs because the outer table only produced a single row to iterate over.

Lab Exercise 2: Triggering Hash Join



The screenshot shows the pgAdmin 4 interface. On the left, the 'Object Explorer' pane shows a database named 'dvdrental' with various tables and indexes. The 'language' table is selected. The main pane displays a SQL query:

```
1 EXPLAIN (ANALYZE, BUFFERS)
2 SELECT f.title, l.name as language
3 FROM film f
4 JOIN language l ON f.language_id = l.language_id;
```

Below the query, the 'Data Output' pane shows the 'QUERY PLAN' for the execution. The plan consists of 11 steps:

- Hash Join (cost=1.14..103.62 rows=1000 width=99) (actual time=0.129..1.092 rows=1000.00 loops=1)
- Hash Cond: (f.language_id = l.language_id)
- Buffers: shared hit=89
- > Seq Scan on film f (cost=0.00..98.00 rows=1000 width=17) (actual time=0.053..0.675 rows=1000.00 loops=1)
- Buffers: shared hit=88
- > Hash (cost=1.06..1.06 rows=6 width=88) (actual time=0.053..0.054 rows=6.00 loops=1)
- Buckets: 1024 Batches: 1 Memory Usage: 9kB
- Buffers: shared hit=1
- > Seq Scan on language l (cost=0.00..1.06 rows=6 width=88) (actual time=0.041..0.044 rows=6.00 loops=1)
- Buffers: shared hit=1
- Planning:

The bottom status bar indicates 'Total rows: 14' and 'Query complete 00:00:00.131'.

Identify Phases:

The smaller table (language) is designated as the inner table to 'build' the hash table in memory. The larger table (film) acts as the outer table and is scanned to 'probe' the hash table for matching keys.

Memory Check:

The hash node displays the specific amount of RAM (Memory Usage) allocated to store the built hash map. 9kB

Hypothesis:

When reducing work_mem to 64kB, the execution plan remains unchanged, still showing Batches. This occurs because the inner table (language) is extremely small, containing only 6 rows. The total memory required to build the Hash Table is merely 9kB. Since 9kB is strictly less than the new 64kB limit, the dataset still easily fits entirely within RAM, successfully avoiding any disk spill or I/O bottleneck.

Lab Exercise 3: Triggering Merge Join

The screenshot shows the pgAdmin 4 interface with a query window open. The query is as follows:

```

1 SET enable_hashjoin = OFF;
2 SET enable_nestloop = OFF;
3
4 EXPLAIN (ANALYZE, BUFFERS)
5 SELECT f.film_id, f.title, i.inventory_id
6 FROM film f
7 JOIN inventory i ON f.film_id = i.film_id
8 ORDER BY f.film_id;

```

The query plan output is shown below:

Step	Operation	Cost	Rows	Width	Actual Time	Actual Rows	Actual Width	Loops
1	Merge Join	(cost=349.64..551.02 rows=4581 width=23)	4581	23	2.224..4.201	4581.00		1
2	Merge Cond: (f.film_id = i.film_id)							
3	Buffers: shared hit=113							
4	-> Index Scan using film_pkey on film f	(cost=0.28..130.44 rows=1000 width=19)	1000	19	0.029..0.871	1000.00		1
5	Index Searches: 1							
6	Buffers: shared hit=85							
7	-> Sort (cost=349.37..360.82 rows=4581 width=6)	(cost=349.37..360.82 rows=4581 width=6)	4581	6	2.181..2.396	4581.00		1
8	Sort Key: i.film_id							

Structure:

An explicit Sort node is typically absent because the system uses an Index scan on `film_id`. Because B-Tree indexes are natively sorted, the Optimizer retrieves the sorted data for free, entirely bypassing the expensive mathematical sorting phase.

Compare:

When run `RESET ALL` and remove the `ORDER BY` clause, the Optimizer reverts to a Hash join. Without the strict constraint of returning ordered data, a Hash join evaluates as the most cost-effective algorithm ($O(N+M)$) for joining large, unfiltered datasets.

Hands-on Lab: Optimizing dvdrental

Task 1: Influence the Optimizer

pgAdmin 4

File Object Tools Edit View Window Help

Object Explorer: Servers (1) PostgreSQL 18 Databases (2) dvdrental Schemas (1) public

Query: Query History

```
1 -- Force a Sequential Scan by disabling Index Scans
2 SET enable_indexscan = off;
3 EXPLAIN ANALYZE SELECT * FROM rental WHERE rental_date > '2005-05-24';
4
5 --Re-enable and compare the Cost and Time
6 SET enable_indexscan = on;
7 EXPLAIN ANALYZE SELECT * FROM rental WHERE rental_date > '2005-05-24';
```

Data Output Messages Notifications

Showing rows: 1 to 7 Page No: 1 of 1

QUERY PLAN
1 Seq Scan on rental (cost=0.00..350.55 rows=16044 width=36) (actual time=0.073..10.094 rows=16047.00 loo...
2 Filter: (rental_date > '2005-05-24 00:00:00':timestamp without time zone)
3 Buffers: shared hit=150
4 Planning:
5 Buffers: shared hit=87 read=1
6 Planning Time: 29.193 ms
7 Execution Time: 11.666 ms

Total rows: 7 Query complete 00:00:00.146 CRLF Ln 2, Col 1

pgAdmin 4

File Object Tools Edit View Window Help

Object Explorer: Servers (1) PostgreSQL 18 Databases (2) dvdrental Schemas (1) public

Query: Query History

```
1 -- Force a Sequential Scan by disabling Index Scans
2 SET enable_indexscan = off;
3 EXPLAIN ANALYZE SELECT * FROM rental WHERE rental_date > '2005-05-24';
4
5 --Re-enable and compare the Cost and Time
6 SET enable_indexscan = on;
7 EXPLAIN ANALYZE SELECT * FROM rental WHERE rental_date > '2005-05-24';
```

Data Output Messages Notifications

Showing rows: 1 to 7 Page No: 1 of 1

QUERY PLAN
1 Seq Scan on rental (cost=0.00..350.55 rows=16044 width=36) (actual time=0.022..4.651 rows=16047.00 loops=1)
2 Filter: (rental_date > '2005-05-24 00:00:00':timestamp without time zone)
3 Buffers: shared hit=150
4 Planning:
5 Buffers: shared hit=3
6 Planning Time: 0.202 ms
7 Execution Time: 6.288 ms

Total rows: 7 Query complete 00:00:00.162 CRLF Ln 5, Col 42

Task 2: Join Strategy Analysis

The screenshot shows the pgAdmin 4 interface with a query window open. The query is as follows:

```
1 EXPLAIN ANALYZE
2 SELECT c.first_name, r.rental_date, i.inventory_id
3 FROM customer c
4 JOIN rental r ON c.customer_id = r.customer_id
5 JOIN inventory i ON r.inventory_id = i.inventory_id;
```

The 'Query Plan' tab is selected, displaying the following execution plan:

Step	Operation	Cost	Rows	Width	Actual Time	Actual Rows	Actual Width	Loops
1	Hash Join	150.55..545.57	16044	18	4.744..44.926	16047.00		1
2	Hash Cond: (r.inventory_id = i.inventory_id)							
3	Buffers: shared hit=184							
4	-> Hash Join	22.48..375.33	16044	18	0.730..22.391	16047.00		1
5	Hash Cond: (r.customer_id = c.customer_id)							
6	Buffers: shared hit=159							
7	-> Seq Scan on rental r	0.00..310.44	16044	14	0.048..3.432	16047.00		
8	Buffers: shared hit=150							
9	-> Hash	14.99..14.99	599	10	0.657..0.659	599.00		1
10	Buckets: 1024 Batches: 1 Memory Usage: 34kB							

Summary statistics at the bottom: Total rows: 22, Query complete 00:00:00.223, CRLF, Ln 4, Col 47.

The execution plan clearly demonstrates that the Optimizer utilized the Hash Join strategy to perform the 3-way join, successfully nesting one Hash Join operation inside another.

The query joins three entire tables (customer, rental, inventory) without any restrictive WHERE filters, requiring the system to process a massive volume of data.